
PROJECT PROPOSAL DOCUMENT

Restaurant Reservation System

Design Patterns Project — Week 1

#	Student Full Name	Group
1	Youssef Mohamed Salem	SE1
2	Omar Mohamed Helmy	SE1
3	Fahmy Mahmoud	SE1
4	Ziad Haitham Yehia	SE3

2. Project Title

Restaurant Reservation System

A full-stack web application that enables customers to discover restaurants, check real-time table availability, and book reservations online. Restaurant administrators manage their tables, time slots, and bookings through a dedicated management panel, while a super-admin oversees the entire platform.

3. Problem Statement

Traditional restaurant reservation processes rely heavily on phone calls, walk-ins, or manual logbooks. This leads to several persistent problems that affect both customers and restaurant staff:

- Customers have no way to check real-time table availability before visiting or calling.
- Phone-based bookings are inefficient, error-prone, and unavailable outside business hours.
- Restaurant staff spend significant time managing reservations manually, leading to double-bookings and scheduling conflicts.
- There is no centralized system for customers to view, modify, or cancel their reservations conveniently.
- Restaurant owners lack analytical insights into booking trends, peak hours, and table utilization.

The Restaurant Reservation System addresses all of these pain points by providing a centralized, digital platform that automates the reservation lifecycle — from discovery and booking to confirmation, cancellation, and reporting.

4. Project Objectives

The system is designed to achieve the following objectives:

- Provide a secure, role-based authentication system (Customer, Restaurant Admin) using JSON Web Tokens (JWT).
- Allow restaurant administrators to manage their restaurant profile, tables, and available time slots through a dedicated admin interface.
- Enable customers to search for restaurants by name, city, or cuisine type, and check real-time table availability for a chosen date and guest count.
- Implement an intelligent table auto-assignment mechanism that selects the most suitable table based on party size and availability.
- Support a full reservation lifecycle: create → confirm → cancel, with business rules.
- Deliver in-app notifications to customers upon booking confirmation and cancellation.
- Apply software design patterns (Strategy, Observer, Facade) in a practical, meaningful way within the codebase.
- Provide restaurant admins with booking reports: today's reservations, upcoming bookings, and popular time slots.

5. Target Users

User Type	Description	System Role
Customer	Individuals who browse restaurants and make table reservations online.	CUSTOMER
Restaurant Admin	Restaurant owners or managers who manage their restaurant, tables, time slots, and bookings.	RESTAURANT_ADMIN

6. Technologies and Frameworks

Layer	Technology / Framework	Purpose
Backend	Java 17 + Spring Boot 3.x	Core application framework. Provides REST API, dependency injection, and layered architecture.
Security	Spring Security + JWT	Stateless authentication and role-based access control.
Database	MySQL	Relational database for storing users, restaurants, tables, reservations, and notifications.
ORM	Spring Data JPA	Object-relational mapping. Manages all database interactions without raw SQL.
Frontend	Angular	Single-page application framework. Communicates with the backend via REST API using HttpClient.
Version Control	Git + GitHub	Source control, branching strategy, and team collaboration.
IDE	IntelliJ IDEA	Primary development environment for Java and Spring Boot.

7. Expected Design Patterns

Pattern	Where Applied	Problem It Solves
Strategy Pattern	TableAssignmentService — table selection algorithm	Different algorithms are needed to assign a table to a reservation (e.g., Best Fit selects the smallest sufficient table; First Available selects the first free table). The Strategy pattern allows switching between these algorithms at runtime without modifying the reservation logic.
Observer Pattern	ReservationEventListener — post-booking notifications	When a reservation is created or cancelled, the NotificationService must be triggered automatically. The Observer pattern decouples ReservationService from NotificationService, so new reactions (e.g., email, SMS) can be added without changing existing code.
Singleton Pattern		Each service or repository must have exactly one shared instance across the entire application to avoid wasted memory and inconsistent state.

8. Initial High-Level System Overview

System Architecture

The backend follows a five-layer Layered Architecture (N-Tier), ensuring clear separation of concerns, testability, and maintainability:

Layer	Responsibility
Controller	Receives HTTP requests from the Angular frontend. Validates request DTOs. Delegates work to the Service layer. Returns structured ResponseEntity responses.
Service	Contains all business logic: availability checking, auto table assignment, cancellation rules, and notification dispatch.
Repository	Spring Data JPA interfaces that handle all database queries. No raw SQL is used.
Entity	Java classes annotated with @Entity that map directly to MySQL database tables.
DTO	Data Transfer Objects used to shape API request and response payloads, decoupling the API contract from the internal database schema.

Core Database Entities

Entity	Table Name	Key Attributes
User	users	id, fullName, email, password (hashed), phone, role (CUSTOMER / RESTAURANT_ADMIN / SUPER_ADMIN), createdAt
Restaurant	restaurants	id, name, description, address, city, cuisineType, openingTime, closingTime, isActive, adminId (FK)
RestaurantTable	restaurant_tables	id, restaurantId (FK), tableNumber, capacity, location (INDOOR / OUTDOOR / TERRACE), isAvailable
TimeSlot	time_slots	id, restaurantId (FK), slotTime, dayOfWeek, isActive
Reservation	reservations	id, customerId (FK), restaurantId (FK), tableId (FK), timeSlotId (FK), reservationDate, guestCount, status (PENDING / CONFIRMED / CANCELLED), specialRequest, createdAt
Notification	notifications	id, userId (FK), message, type (BOOKING_CONFIRMED / BOOKING_CANCELLED), isRead, createdAt

Key System Features

- Role-based access control: Customers book, Admins manage oversees everything.
- Real-time availability engine: checks time slots, table capacity, and existing reservations to return only genuinely available options.
- Intelligent auto-assignment: the system automatically selects the best table based on guest count using the Strategy pattern.
- Full reservation lifecycle management: PENDING → CONFIRMED → CANCELLED with business rule enforcement.
- In-app notification system: customers receive confirmation and cancellation notifications stored in the database.
- Angular frontend: responsive single-page application communicating with the REST API via Angular HttpClient and Guards for route protection.